

Networking

Use Case 5: Secure Network Architecture

Scenario: Design a VPC for a multi-tier web application.

Tasks:

- o Create subnets (public and private).
- o Configure NAT Gateways for private subnets.
- o Use security groups and NACLs for access control.
- o Test connectivity between tiers.

Step 1: Plan the VPC Architecture

The architecture will consist of:

1. **Public Subnet:** For the web servers, accessible from the internet.

2. **Private Subnet:** For the application and database servers, isolated from the internet.
3. **NAT Gateway:** To allow private subnet resources to access the internet for updates or downloads without exposing them.

Key Components:

- . **Public Subnet:** Hosts resources that must be accessible from the internet (e.g., load balancers).
 - . **Private Subnet:** Hosts internal resources like app servers and databases.
 - . **NAT Gateway:** Facilitates secure outbound traffic from private subnets.
 - . **Security Groups and Network ACLs:** Control inbound and outbound traffic.
-

Step 2: Create a VPC

1. Open the **AWS Management Console**.
 2. Navigate to **VPC > Your VPCs** and click **Create VPC**.
 - **Name tag:** MyVPC.
 - **IPv4 CIDR block:** Choose a block like 10.0.0.0/16
 - **IPv6 CIDR block:** Optional.
 - **Tenancy:** Default.
-

Step 3: Create Subnets

1. **Public Subnet:**
 - Navigate to **Subnets** and click **Create subnet**.
 - Select MyVPC and provide:
 - **Subnet name:** PublicSubnet.

- **Availability Zone:** Choose one (e.g., ap-south-1a).
 - **IPv4 CIDR block:** 10.0.1.0/24.
 - Enable **Auto-assign public IP**.
 - 2. **Private Subnet:**
 - Create another subnet for private resources:
 - **Subnet name:** PrivateSubnet.
 - **Availability Zone:** Same or different from the public subnet (e.g., ap-south-1b).
 - **IPv4 CIDR block:** 10.0.2.0/24.
-

Step 4: Configure an Internet Gateway

1. Go to **Internet Gateways** and click **Create internet gateway**.
 - **Name tag:** MyIGW.
 - Attach it to MyVPC.

2. Update the public subnet route table:
 - Go to **Route Tables**, find the one associated with the public subnet.
 - Add a route:
 - **Destination:** 0.0.0.0/0.
 - **Target:** Internet Gateway (MyIGW).
-

Step 5: Configure a NAT Gateway for Private Subnets

1. **Create an Elastic IP (EIP):**
 - Navigate to **Elastic IPs**, allocate a new address, and tag it as MyNATGatewayEIP.
2. **Create the NAT Gateway:**
 - Go to **NAT Gateways** and click **Create NAT Gateway**.
 - Attach it to the **PublicSubnet**.

- Assign the **Elastic IP (MyNATGatewayEIP)**.
 - 3. **Update the Private Subnet Route Table:**
 - Find the route table for the private subnet and add:
 - **Destination:** 0.0.0.0/0.
 - **Target:** NAT Gateway (MyNATGateway).
-

Step 6: Configure Security Groups

1. **Web Server Security Group (SG-Web):**
 - Allow HTTP/HTTPS traffic from the internet:
 - **Inbound rules:**
 - **Type:** HTTP, HTTPS
 - **Source:** 0.0.0.0/0 or restrict to specific IP ranges.

- Allow SSH traffic from trusted IPs only.
- **Outbound rules:**
 - Allow all traffic (0.0.0.0/0) for updates, if required.

2. **Application Server Security Group (SG-App):**

- Allow traffic only from the web server (SG-Web):
 - **Inbound rules:**
 - **Type:** Custom TCP (Application Port, e.g., 8080)
 - **Source:** SG-Web.
- Allow outbound traffic for internet access via NAT Gateway.

3. **Database Server Security Group (SG-DB):**

- Allow traffic only from the application server (SG-App):

- **Inbound rules:**
 - **Type:** MySQL/Aurora (or relevant DB port).
 - **Source:** SG-App.
 - Allow outbound traffic as required.
-

Step 7: Configure Network ACLs

1. **Public Subnet NACL:**
 - **Inbound rules:**
 - Allow HTTP/HTTPS traffic from 0.0.0.0/0.
 - Allow ephemeral ports (1024-65535) for return traffic.
 - **Outbound rules:**
 - Allow all traffic.
2. **Private Subnet NACL:**
 - **Inbound rules:**

- Allow traffic from the web server and application server security groups.
 - Allow ephemeral ports for return traffic.
 - **Outbound rules:**
 - Allow all traffic for internet access via the NAT Gateway.
-

Step 8: Test Connectivity

1. Test Public Access:

- Deploy a simple web application in the public subnet.
- Test HTTP/HTTPS access from a browser.

2. Test Private Access:

- Deploy a sample application server in the private subnet.

- Ensure it communicates with the web server and database server.
 - 3. **Ping Tests:**
 - Use SSH to access the public subnet and test connectivity to private subnet resources.
 - 4. **Simulate Real Traffic:**
 - Use tools like **Apache Bench** or **Postman** to simulate traffic and monitor performance.
-

Security Optimization

- Use **AWS WAF** to filter malicious traffic to the web servers.
- Enable **VPC Flow Logs** to monitor and log network traffic.
- Use **AWS Shield** for DDoS protection.
- Restrict access using **IAM policies** to enforce least privilege.

Cost Optimization

- Use **Elastic Load Balancers (ELBs)** for traffic distribution instead of running multiple public instances.
- Enable **Auto-scaling** for application servers to manage resource utilization efficiently.
- Review **AWS Cost Explorer** periodically to identify cost-saving opportunities.